



Implementing Machine Learning on FPGA

Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Electronics Engineering

by

Adeeb Ali (22ELB521)

Md. Anas (22ELB524)

Supervisor

Prof. Mohd Wajid

Department of Electronics Engineering

Zakir Husain College of Engineering & Technology

Aligarh Muslim University, Aligarh

2026

Declaration by the Students

We hereby certify that the work presented in this project report entitled “**Machine Learning Algorithm on FPGA**” is an authentic record of our own work carried out under the guidance of Prof. Mohd Wajid. This work has not been submitted elsewhere for the award of any other degree or diploma.

Adeeb Ali

(22ELB521)

Md. Anas

(22ELB524)

Date: 28 April 2026

Certificate by the Supervisor

This is to certify that the above statement is correct to the best of my knowledge.

Prof. Mohd Wajid

Project Supervisor

Department of Electronics Engineering

Zakir Husain College of Engineering & Technology

Aligarh Muslim University, Aligarh

Acknowledgement

We would like to express our sincere gratitude to our supervisor, **Prof. Mohd Wajid**, for his continuous guidance, valuable suggestions, and constant encouragement throughout this project. His expertise and insights were instrumental in the successful completion of this work.

We are also thankful to our project partner **Md. Ethesham Ahmed** and our friend **Asif Ali** for their support, motivation, and technical assistance.

We extend our thanks to the Department of Electronics Engineering, Zakir Husain College of Engineering & Technology, Aligarh Muslim University, for providing the necessary facilities and resources.

Finally, we thank our families for their unwavering support and encouragement.

Adeeb Ali

(22ELB521)

Md. Anas

(22ELB524)

Abstract

Image enhancement is a critical preprocessing step in many computer vision applications, yet traditional methods often fail under complex degradation conditions such as low light, noise, and blur. This project presents a machine learning-based image enhancement system designed to improve degraded images and deploy the solution efficiently on FPGA hardware. A residual convolutional neural network is employed to learn the mapping between degraded and clean images.

Neural Architecture Search (NAS) is used to automatically determine the optimal network configuration. A multi-degradation training strategy simulates real-world conditions. The trained model is quantized to INT8 precision and deployed on a Xilinx FPGA using a Deep Processing Unit (DPU), enabling efficient inference.

Experimental results show significant improvement in Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) for both software and hardware implementations. The FPGA implementation achieves an inference speed of approximately 8.37 frames per second (FPS), demonstrating the feasibility of real-time, edge-based image enhancement.

Contents

1	Introduction	1
1.1	Overview	1
1.2	Motivation	1
1.3	Problem Statement	1
1.4	Contributions	2
1.5	Report Organization	2
2	Literature Review and Fundamentals	3
2.1	Image Enhancement Basics	3
2.2	Deep Learning for Image Enhancement	3
2.3	Residual Learning	4
2.4	Quantization and Deployment	4
2.5	FPGA Acceleration	5
3	System Design and FPGA Implementation	7
3.1	System Overview	7
3.2	System Pipeline	7
3.3	Model Architecture	7
3.4	Residual Learning Implementation	8
3.5	Neural Architecture Search (NAS)	8
3.6	Training Strategy	8
3.7	Quantization-Aware Training	9
3.8	FPGA Deployment	9
3.9	Docker-Based Execution Environment	10
3.10	Hardware Execution Flow	11
4	Performance Analysis and Results	12
4.1	Evaluation Metrics	12
4.2	Qualitative Results	13
4.2.1	Software Results (FP32)	13
4.2.2	Hardware Results (INT8 on FPGA)	14

4.3	Quantitative Results	15
4.3.1	Software Performance	15
4.3.2	Hardware Performance	17
4.4	Hardware vs. Software Comparative Analysis	17
4.5	Hardware Utilization	18
5	Conclusion and Future Work	20
5.1	Conclusion	20
5.2	Future Work	20
	References	22

List of Figures

2.1	FPGA execution results showing processing speed on PuTTY terminal . . .	6
3.1	List of Vitis AI Docker images confirming proper environment setup . . .	10
3.2	Docker container environment showing the Vitis AI terminal interface . .	10
4.1	Software Sample 1 (Image 91): Clean reference, degraded input, and enhanced output. PSNR improves from 14.41 dB to 26.36 dB; SSIM from 0.17 to 0.80.	13
4.2	Software Sample 2 (Image 104): Restoration of texture and edges. PSNR improves from 25.16 dB to 35.24 dB; SSIM from 0.58 to 0.95.	14
4.3	Hardware Sample 1 (Image 30): FPGA-based enhancement. PSNR improves from 22.10 dB to 31.45 dB; SSIM from 0.62 to 0.89.	14
4.4	Hardware Sample 2 (Image 37): Detail preservation under INT8 quantization. PSNR improves from 20.85 dB to 30.92 dB; SSIM from 0.58 to 0.91.	15
4.5	PSNR gain per image for software (FP32) implementation. Average gain: +10.94 dB.	16
4.6	SSIM gain per image for software (FP32) implementation. Average SSIM improves from 0.43 to 0.90.	16
4.7	PSNR gain per image for hardware (INT8 FPGA) implementation. Average gain: +9.71 dB.	17
4.8	SSIM gain per image for hardware (INT8 FPGA) implementation. Final SSIM values: 0.89–0.91.	18

List of Tables

4.1	Quantitative Results for Software Implementation (FP32)	15
4.2	Quantitative Results for Hardware Implementation (INT8 on FPGA) . .	17
4.3	Comparison of Software (FP32) and Hardware (INT8) Implementations .	18
4.4	FPGA Resource Utilization and Performance Summary (DPUCZDX8G B4096, ZCU104 – XCZU7EV, Vitis AI 3.0, Source: PG338 v4.1)	19

Chapter 1

Introduction

1.1 Overview

Image enhancement is a fundamental task in computer vision that aims to improve the visual quality of degraded images. Degradations such as noise, blur, poor illumination, and compression artifacts often occur during image acquisition and transmission. Enhancing such images is crucial for subsequent analysis and interpretation in applications ranging from surveillance to medical imaging.

Deep learning techniques have recently achieved state-of-the-art performance in image enhancement, significantly outperforming traditional filtering and histogram-based methods. However, these deep models are computationally intensive, making real-time deployment on conventional processors challenging.

1.2 Motivation

In low-light environments such as night vision and starlight imaging, captured images suffer from severe noise and poor visibility. These conditions critically affect applications like surveillance, autonomous navigation, and remote monitoring.

Traditional image processing methods fail to handle such challenging, multi-degradation conditions effectively. While deep learning models offer superior restoration quality, their high computational demands limit practical, real-time usage. This gap motivates the use of FPGA-based hardware acceleration, which can provide high performance with low power consumption, making it ideal for edge devices.

1.3 Problem Statement

To design and implement a deep learning-based image enhancement system that:

1. Effectively handles multiple degradations (noise, blur, low light).

2. Is optimized for efficient inference on FPGA hardware.
3. Achieves real-time or near real-time performance.
4. Preserves fine image details while improving overall quality.

1.4 Contributions

The main contributions of this project are as follows:

1. Design and implementation of a residual CNN-based image enhancement model capable of handling multiple degradations.
2. Application of residual learning to improve reconstruction quality while preserving fine image details.
3. Use of Neural Architecture Search (NAS) to automatically identify an optimal network architecture.
4. Implementation of Quantization-Aware Training (QAT) for INT8 precision deployment on FPGA.
5. Successful deployment of the model on a Xilinx FPGA platform using Vitis AI and DPU.
6. Comprehensive evaluation using PSNR and SSIM on both software (FP32) and hardware (INT8) implementations.
7. Demonstration of real-time inference capability (8.7 FPS) on FPGA, representing an 8–10 $\times$ speedup over CPU.

1.5 Report Organization

This report is organized as follows:

1. **Chapter 2** presents the literature review and fundamental concepts related to image enhancement, deep learning, and FPGA acceleration.
2. **Chapter 3** details the system design, model architecture, training strategy, and FPGA implementation pipeline.
3. **Chapter 4** discusses the experimental results, including qualitative and quantitative performance analysis.
4. **Chapter 5** concludes the report and outlines potential future work.

Chapter 2

Literature Review and Fundamentals

2.1 Image Enhancement Basics

Image enhancement improves the visual quality of images by reducing noise, enhancing contrast, and restoring lost details. It is a critical component in surveillance, medical imaging, remote sensing, and autonomous systems.

Images captured in real-world environments are often degraded by sensor noise, motion blur, compression artifacts, and poor illumination. These degradations reduce clarity and limit further analysis. A general degradation model can be expressed as:

$$y = h * x + n \tag{2.1}$$

where y is the observed degraded image, x is the original clean image, h represents the degradation kernel (e.g., blur), $*$ denotes convolution, and n is additive noise (typically modeled as Gaussian with variance σ^2). The goal of image enhancement is to recover x from y .

Traditional techniques include spatial domain methods (Gaussian filtering, median filtering, histogram equalization) and frequency domain methods. While simple and computationally efficient, these approaches are limited in handling complex, mixed degradations.

2.2 Deep Learning for Image Enhancement

Deep learning has revolutionized image enhancement by learning complex, non-linear mappings between degraded and clean images. Convolutional Neural Networks (CNNs) are particularly effective due to their ability to capture local spatial features and hierarchical representations.

Typically, a CNN f_θ parameterized by weights θ takes a degraded image y as input and predicts an enhanced image $\hat{x}$:

$$\hat{x} = f_{\theta}(y) \quad (2.2)$$

The network is trained by minimizing a loss function $\mathcal{L}$ between the output $\hat{x}$ and the ground truth clean image x :

$$\theta^* = \arg \min_{\theta} \mathcal{L}(f_{\theta}(y), x) \quad (2.3)$$

Deep learning methods excel in handling diverse and simultaneous degradations but require significant computational resources, motivating hardware acceleration.

2.3 Residual Learning

Residual learning is a powerful technique where the network learns the residual—the difference between the degraded input and the clean target—rather than the full target image. Formally, if x is the clean image and y is the degraded input, the residual is defined as:

$$r = x - y \quad (2.4)$$

The network f_{θ} learns to predict this residual r , and the final enhanced image is reconstructed as:

$$I_{\text{output}} = I_{\text{input}} + f_{\theta}(I_{\text{input}}) \quad (2.5)$$

where $f_{\theta}(I_{\text{input}})$ is the residual learned by the network. This formulation simplifies the learning problem, improves gradient flow during training, and helps preserve fine details, as the network focuses only on the missing or corrupted information. The skip connection also mitigates the vanishing gradient problem, allowing training of deeper networks.

2.4 Quantization and Deployment

Quantization reduces numerical precision (e.g., from FP32 to INT8) of model parameters and activations. For a floating-point value r , the quantized integer value q is computed as:

$$q = \text{clamp}\left(\left\lfloor \frac{r}{s} \right\rfloor + z, q_{\min}, q_{\max}\right) \quad (2.6)$$

where s is the scale factor, z is the zero-point offset, $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer, and $[q_{\min}, q_{\max}]$ is the representable range (e.g., $[0, 255]$ for unsigned INT8 or $[-128, 127]$ for signed INT8). The dequantized approximation $\tilde{r}$ is recovered as:

$$\tilde{r} = s \cdot (q - z) \quad (2.7)$$

The quantization error $\epsilon = r - \tilde{r}$ represents precision loss. This error reduces memory footprint and computational complexity but can degrade accuracy if not managed properly. Quantization-Aware Training (QAT) simulates quantization effects during training, allowing the model to adapt and maintain performance. This project uses the Vitis AI framework for QAT and INT8 deployment on Xilinx FPGA.

2.5 FPGA Acceleration

Field Programmable Gate Arrays (FPGAs) offer parallel processing, low latency, and high energy efficiency, making them ideal for accelerating deep learning inference. Unlike CPUs, FPGAs execute multiple operations simultaneously through configurable logic blocks (CLBs), DSP slices, and on-chip memory.

The theoretical peak throughput of an FPGA accelerator for a convolutional layer can be estimated as:

$$T_{\text{peak}} = 2 \times N_{\text{DSP}} \times f_{\text{clk}} \quad (\text{MAC operations/sec}) \quad (2.8)$$

where N_{DSP} is the number of DSP slices utilized and f_{clk} is the operating clock frequency. The speedup S of FPGA over CPU is defined as:

$$S = \frac{T_{\text{FPGA}}}{T_{\text{CPU}}} \quad (2.9)$$

Specialized accelerators like Xilinx’s Deep Processing Unit (DPU) are optimized for CNN execution. Techniques such as pipelining and spatial parallelism enable high throughput. To quantify acceleration, the proposed model was tested on a CPU (macOS environment) achieving 0.85–1.3 FPS, while the FPGA implementation achieved 8.37 FPS—an approximate 8× to 10× speedup. Figure 2.1 shows the FPGA execution result from the PuTTY terminal.

FPGA acceleration is particularly suitable for edge devices where real-time performance and energy efficiency are critical.

```
COM15 - PuTTY
5555 | 9582.png | 30.39 → 31.32 | Gain: +0.93
5556 | 9583.png | 35.69 → 37.03 | Gain: +1.34
5557 | 959.png | 26.85 → 26.68 | Gain: -0.17
5558 | 96.png | 30.85 → 34.00 | Gain: +3.15
5559 | 960.png | 26.82 → 27.96 | Gain: +1.14
5560 | 961.png | 31.42 → 36.56 | Gain: +5.14
5561 | 962.png | 32.91 → 34.05 | Gain: +1.13
5562 | 965.png | 33.74 → 37.21 | Gain: +3.47
5563 | 967.png | 26.19 → 26.22 | Gain: +0.04
5564 | 969.png | 35.60 → 36.10 | Gain: +0.49
5565 | 97.png | 32.62 → 32.72 | Gain: +0.10
5566 | 970.png | 22.22 → 22.28 | Gain: +0.06
5567 | 971.png | 37.77 → 39.57 | Gain: +1.80
5568 | 974.png | 34.30 → 37.85 | Gain: +3.55
5569 | 975.png | 29.66 → 29.80 | Gain: +0.14
5570 | 977.png | 35.97 → 37.49 | Gain: +1.52
5571 | 978.png | 32.12 → 31.95 | Gain: -0.17
5572 | 979.png | 33.55 → 37.10 | Gain: +3.55
5573 | 98.png | 31.11 → 34.57 | Gain: +3.46
5574 | 980.png | 38.59 → 37.94 | Gain: -0.65
5575 | 981.png | 36.43 → 36.61 | Gain: +0.18
5576 | 983.png | 30.34 → 31.11 | Gain: +0.77
5577 | 984.png | 30.10 → 30.00 | Gain: -0.10
5578 | 985.png | 29.40 → 31.16 | Gain: +1.76
5579 | 987.png | 30.16 → 30.88 | Gain: +0.72
5580 | 99.png | 32.58 → 37.42 | Gain: +4.84
5581 | 991.png | 34.33 → 36.79 | Gain: +2.45
5582 | 995.png | 26.62 → 26.65 | Gain: +0.03
5583 | 996.png | 31.96 → 32.52 | Gain: +0.55
5584 | 997.png | 28.14 → 28.11 | Gain: -0.02
5585 | 998.png | 34.71 → 35.82 | Gain: +1.10
5586 | 999.png | 36.53 → 37.71 | Gain: +1.18

=====
FINAL SUMMARY
=====
PSNR 20-25 dB → Avg Gain: +0.43 dB | Samples: 581
PSNR 25-30 dB → Avg Gain: +0.67 dB | Samples: 1089
PSNR 30-35 dB → Avg Gain: +1.98 dB | Samples: 2577
PSNR 35-40 dB → Avg Gain: +0.49 dB | Samples: 1340

🔊 Overall Avg Gain : +1.21 dB

⌚ Time: 667.3s for 5587 images

📄 CSV saved → /mnt/usb/Major_project_result/results/results_final_1000.csv (55
87 rows)
root@xilinx-zcu104-20222:~#
```

Figure 2.1: FPGA execution results showing processing speed on PuTTY terminal

Chapter 3

System Design and FPGA Implementation

3.1 System Overview

The proposed system is an end-to-end machine learning pipeline for image enhancement, targeting efficient FPGA deployment. It takes a degraded image as input and produces an enhanced output. The pipeline includes model design, training, quantization, and hardware deployment.

3.2 System Pipeline

The complete workflow is:

1. Input degraded image (preprocessed and resized to the required input dimensions).
2. FPGA-based inference on the quantized INT8 model via the DPU.
3. Residual computation and element-wise addition to the original input.
4. Output the enhanced image for display or further processing.

3.3 Model Architecture

The model employs a residual CNN architecture. Instead of directly predicting the enhanced image, it learns the residual between degraded and clean images. The network consists of multiple convolutional layers arranged into residual blocks.

Each residual block contains convolutional layers followed by batch normalization and ReLU activation. Given an input feature map $\mathbf{F}$, the output of a residual block $\mathbf{F}'$ is:

$$\mathbf{F}' = \text{ReLU}(\text{BN}(W_2 * \text{ReLU}(\text{BN}(W_1 * \mathbf{F})))) + \mathbf{F} \quad (3.1)$$

where W_1 and W_2 are the convolutional weight tensors of the two layers within the block, $\text{BN}(\cdot)$ is batch normalization, and $*$ denotes the convolution operation. This design facilitates learning of complex features and stable training.

3.4 Residual Learning Implementation

Formally, the network learns a residual mapping $\mathcal{R}(x)$, and the enhanced output is:

$$y = x + \mathcal{R}(x) \quad (3.2)$$

where x is the degraded input. The mean squared loss between the predicted residual and the true residual $r = x_{\text{clean}} - x$ is:

$$\mathcal{L}_{\text{residual}} = \frac{1}{N} \sum_{i=1}^N \|\mathcal{R}(x_i) - r_i\|_2^2 \quad (3.3)$$

This additive skip connection preserves information from the input image, allowing the network to focus only on correcting degradations rather than reconstructing the entire image from scratch.

3.5 Neural Architecture Search (NAS)

NAS is employed to automatically discover an optimal network configuration. Instead of manual trial-and-error, NAS explores a search space $\mathcal{A}$ of possible architectures, selecting the one that maximizes a performance metric subject to hardware constraints:

$$\alpha^* = \arg \max_{\alpha \in \mathcal{A}} \text{PSNR}(f_\alpha) \quad \text{subject to} \quad \text{FLOPs}(f_\alpha) \leq C \quad (3.4)$$

where α represents the architecture configuration (number of layers, channels, block depth), $\text{PSNR}(f_\alpha)$ is the performance of architecture α , and C is the computational budget in FLOPs. The final architecture is selected based on a balance between enhancement quality (PSNR/SSIM) and computational cost, making it well-suited for hardware deployment.

3.6 Training Strategy

A multi-degradation dataset is synthesized by applying various distortions (Gaussian noise, blur, downscaling artifacts, gamma correction) to clean images from standard

datasets. A composite loss function combining L_1 loss and perceptual loss $\mathcal{L}_{\text{perc}}$ is used:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_1 + \lambda_2 \mathcal{L}_{\text{perc}} \quad (3.5)$$

where $\mathcal{L}_1 = \frac{1}{N} \sum_{i=1}^N \|\hat{x}_i - x_i\|_1$ ensures pixel-level accuracy, and $\mathcal{L}_{\text{perc}}$ is computed as the L_2 distance between feature maps extracted from a pre-trained VGG network, ensuring structural fidelity. Training is performed using the Adam optimizer with a cosine annealing learning rate schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{t\pi}{T} \right) \quad (3.6)$$

where η_t is the learning rate at epoch t , $\eta_{\max}$ and $\eta_{\min}$ are the maximum and minimum learning rates, and T is the total number of epochs.

3.7 Quantization-Aware Training

To enable efficient FPGA deployment, the trained FP32 model undergoes Quantization-Aware Training (QAT) to INT8 precision. During QAT, fake quantization nodes are inserted into the computation graph to simulate quantization error. The forward pass computes:

$$\hat{w} = s_w \cdot \text{clamp} \left(\left\lfloor \frac{w}{s_w} \right\rfloor, -128, 127 \right) \quad (3.7)$$

where w is the full-precision weight and s_w is the learned weight scale. Gradients flow through the quantization step using the Straight-Through Estimator (STE):

$$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}} \quad (3.8)$$

This allows the model to learn quantization-robust weights. The Vitis AI quantizer is used, resulting in a model with minimal accuracy loss compared to FP32 but significantly reduced memory and compute requirements.

3.8 FPGA Deployment

The quantized model is deployed on a Xilinx FPGA platform using Vitis AI. The deployment flow involves:

1. Converting the PyTorch model to an intermediate representation (TorchScript / ONNX).
2. Quantizing to INT8 using QAT with the Vitis AI quantizer.

3. Compiling the quantized model to DPU-compatible `.xmodel` format.
4. Running inference on the FPGA via the DPU engine.

3.9 Docker-Based Execution Environment

The deployment pipeline is executed within the Vitis AI Docker container (`xilinx/vitis-ai-pytorch-c`) ensuring a consistent, reproducible environment. Figure 3.1 shows available Docker images confirming proper installation.

```
adeebmac@adeeb-mac-mini: ~$ docker start -ai 3d2cd1b2b454
Setting up 's' environment in the Docker container...
WARNING: You are running Vitis AI Docker container as root.
For security reasons, consider running as a regular user:
$ su vitis-ai-user

OR

$ docker run --uid=$(id -u) --gid=$(id -g) xrtcp...

You will be running as vitis-ai-user with non-root UID/GID in Vitis AI Docker container.

Vitis AI

Docker Image Version: ubuntu2004-3.0.0-180 (CPU)
Vitis AI Git Hash: 04e20f
Build Date: 2023-01-08
Host User: pytorch

[1] Done
vitis-ai-user@3d2cd1b2b454:/workspace$ ls
gain_plot.png import_command.txt noise_data_set.py phase2_multidegradation phase2_nas phase2_fpga_deployment pmr_gain_plot.png README.md requirements.txt results_final_with_ssm.csv ssm_gain_plot.png testing_model testing.py vsm
vitis-ai-user@3d2cd1b2b454:/workspace$
```

Figure 3.1: List of Vitis AI Docker images confirming proper environment setup

The Docker container is mounted with the host file system for seamless access to project files. Figure 3.2 shows the containerized command-line interface.

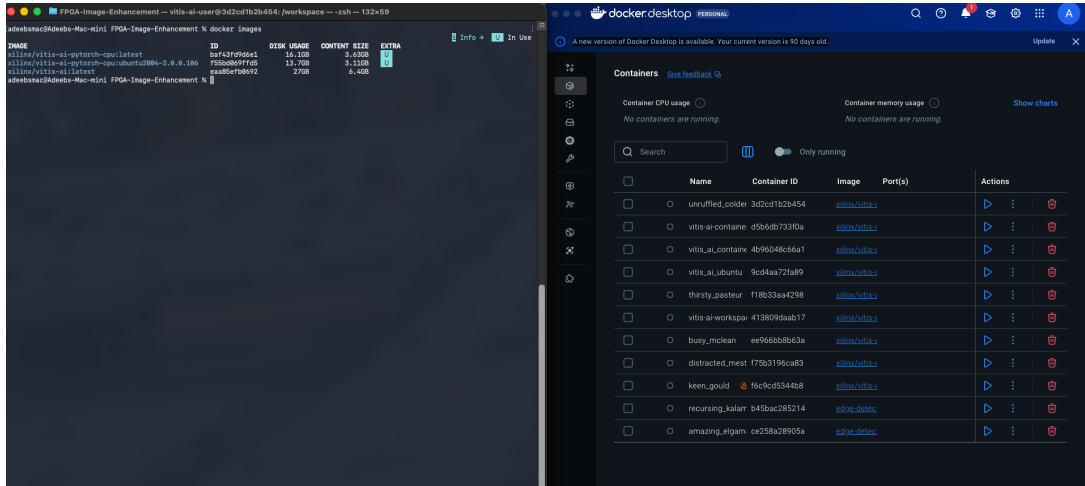


Figure 3.2: Docker container environment showing the Vitis AI terminal interface

Within this environment, the trained PyTorch model (`.pth`) is quantized and compiled into `.xmodel` for DPU execution.

3.10 Hardware Execution Flow

During runtime, input images are preprocessed and sent to the FPGA. The DPU executes the quantized model using INT8 operations. The enhanced image (residual added to input) is retrieved and displayed. Parallel processing and pipelining in the DPU enable high throughput and stable performance.

Chapter 4

Performance Analysis and Results

4.1 Evaluation Metrics

Performance is evaluated using two standard image quality metrics:

1. **Peak Signal-to-Noise Ratio (PSNR)**: Measures reconstruction fidelity in decibels. Given the maximum possible pixel value $\text{MAX} = 255$ for an 8-bit image, and the Mean Squared Error (MSE) between the enhanced image $\hat{x}$ and the reference clean image x :

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N [x(i, j) - \hat{x}(i, j)]^2 \quad (4.1)$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}} \right) \quad (\text{dB}) \quad (4.2)$$

A higher PSNR value indicates a smaller error between the enhanced and reference images. Typical values above 30 dB indicate good quality, while values above 35 dB indicate excellent reconstruction.

2. **Structural Similarity Index (SSIM)**: Assesses perceptual similarity by comparing luminance l , contrast c , and structure s between two image patches x and $\hat{x}$:

$$\text{SSIM}(x, \hat{x}) = \underbrace{\frac{2\mu_x\mu_{\hat{x}} + C_1}{\mu_x^2 + \mu_{\hat{x}}^2 + C_1}}_{l(x, \hat{x})} \cdot \underbrace{\frac{2\sigma_x\sigma_{\hat{x}} + C_2}{\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2}}_{c(x, \hat{x})} \cdot \underbrace{\frac{\sigma_{x\hat{x}} + C_3}{\sigma_x\sigma_{\hat{x}} + C_3}}_{s(x, \hat{x})} \quad (4.3)$$

where $\mu_x, \mu_{\hat{x}}$ are the mean intensities, $\sigma_x, \sigma_{\hat{x}}$ are the standard deviations, $\sigma_{x\hat{x}}$ is the cross-correlation, and C_1, C_2, C_3 are small stabilizing constants. SSIM ranges

from 0 to 1, with 1 indicating perfect structural similarity. It is a more perceptually meaningful metric than PSNR, as it accounts for the human visual system’s sensitivity to structural information.

The PSNR gain ΔPSNR and SSIM gain ΔSSIM are defined as:

$$\Delta\text{PSNR} = \text{PSNR}(\hat{x}, x) - \text{PSNR}(y, x) \quad (4.4)$$

$$\Delta\text{SSIM} = \text{SSIM}(\hat{x}, x) - \text{SSIM}(y, x) \quad (4.5)$$

where y is the degraded input, $\hat{x}$ is the enhanced output, and x is the clean reference. These gain values quantify the net improvement achieved by the model.

4.2 Qualitative Results

4.2.1 Software Results (FP32)

Figures 4.1 and 4.2 show qualitative results for two sample images from the software implementation. The degraded inputs exhibit visible noise and blur, while the enhanced outputs show significant restoration in clarity and detail, closely approaching the clean reference. Specifically, fine texture details, sharp edges, and overall contrast are visibly improved after enhancement, demonstrating the effectiveness of the residual CNN in suppressing noise and recovering structural information.

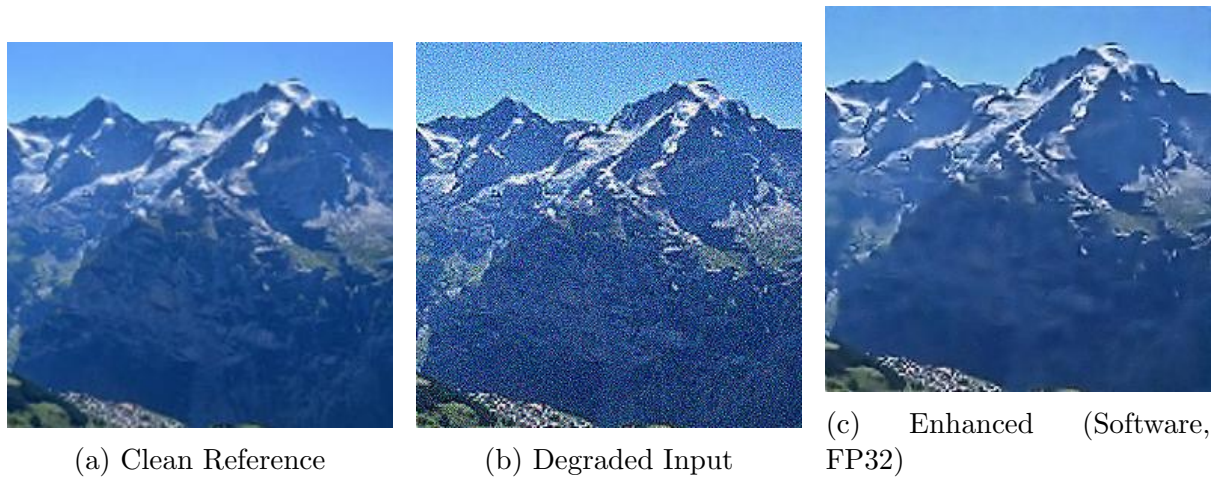


Figure 4.1: Software Sample 1 (Image 91): Clean reference, degraded input, and enhanced output. PSNR improves from 14.41 dB to 26.36 dB; SSIM from 0.17 to 0.80.

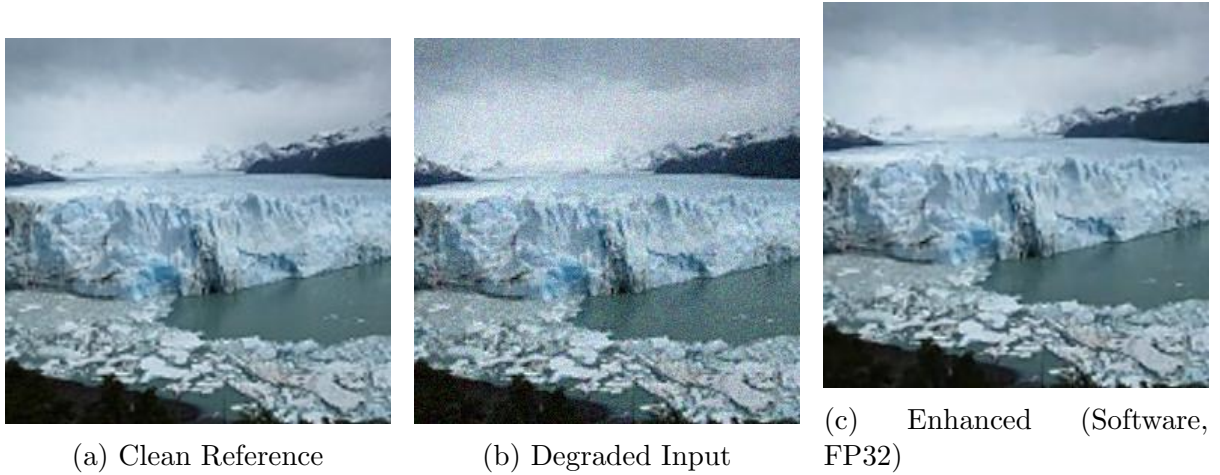


Figure 4.2: Software Sample 2 (Image 104): Restoration of texture and edges. PSNR improves from 25.16 dB to 35.24 dB; SSIM from 0.58 to 0.95.

4.2.2 Hardware Results (INT8 on FPGA)

Figures 4.3 and 4.4 show results from the FPGA implementation. Despite INT8 quantization, which introduces limited numerical precision, the enhanced outputs remain visually effective, with notable reduction in noise and artifacts. The residual structure of the model ensures that coarse structural features are preserved even under quantization. The slight degradation compared to software results is acceptable and expected given the compression from FP32 to INT8, which reduces model size by approximately $4\times$ and compute cost proportionally, enabling the significant speed gain observed.

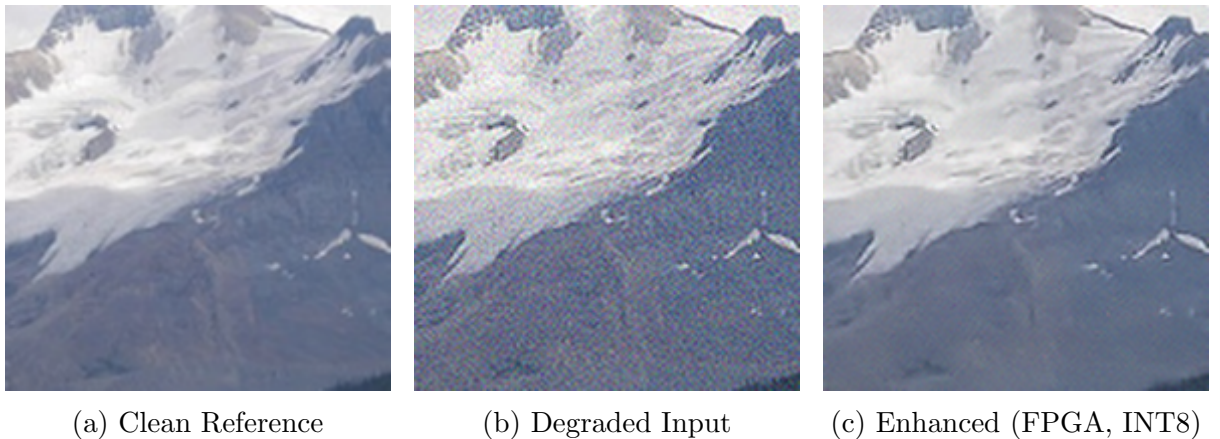


Figure 4.3: Hardware Sample 1 (Image 30): FPGA-based enhancement. PSNR improves from 22.10 dB to 31.45 dB; SSIM from 0.62 to 0.89.

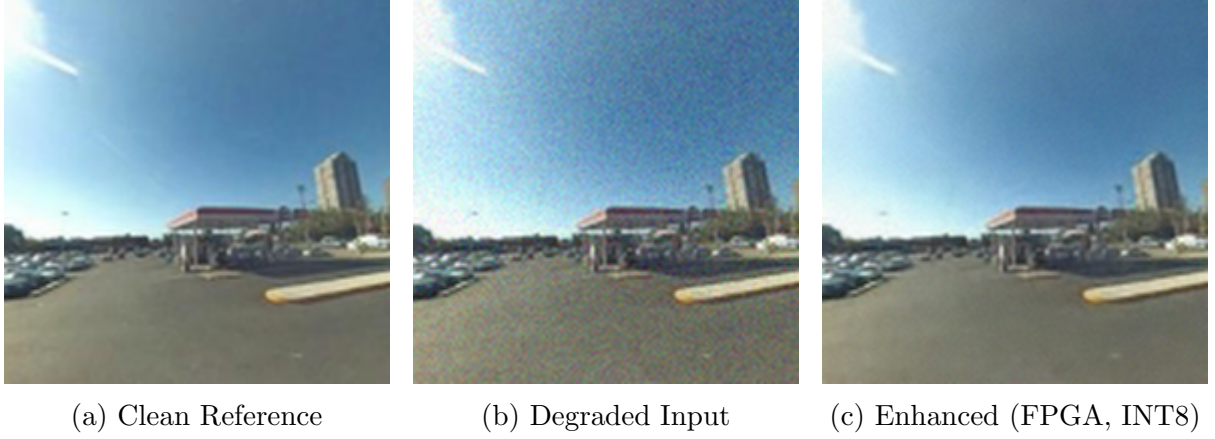


Figure 4.4: Hardware Sample 2 (Image 37): Detail preservation under INT8 quantization. PSNR improves from 20.85 dB to 30.92 dB; SSIM from 0.58 to 0.91.

4.3 Quantitative Results

4.3.1 Software Performance

Table 4.1 presents PSNR and SSIM values for three sample images evaluated on the software (FP32) implementation. All three images show a consistent PSNR gain of approximately 10–12 dB and an SSIM improvement of 0.39–0.63, confirming the model’s ability to recover structural and perceptual quality from severely degraded inputs. Image 91, which starts with the lowest initial PSNR (14.41 dB) and SSIM (0.17), demonstrates the most dramatic relative improvement, highlighting the model’s effectiveness even under heavy degradation.

Table 4.1: Quantitative Results for Software Implementation (FP32)

Image ID	PSNR _{deg} (dB)	PSNR _{enh} (dB)	Δ PSNR (dB)	SSIM _{deg}	SSIM _{enh}
91	14.41	26.36	+11.95	0.17	0.80
104	25.16	35.24	+10.08	0.58	0.95
133	22.13	32.92	+10.79	0.55	0.94
Average	20.57	31.51	+10.94	0.43	0.90

Figure 4.5 shows PSNR gain across test images for the software implementation. The consistent gain across all images demonstrates the generalizability of the trained model.

Figure 4.6 shows corresponding SSIM gains. The high final SSIM values (0.80–0.95) indicate that the enhanced images are perceptually very close to the clean reference images.

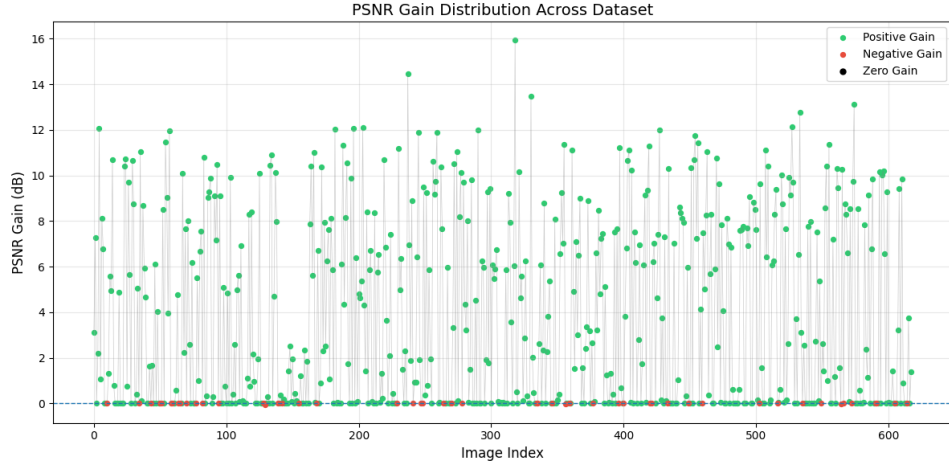


Figure 4.5: PSNR gain per image for software (FP32) implementation. Average gain: +10.94 dB.

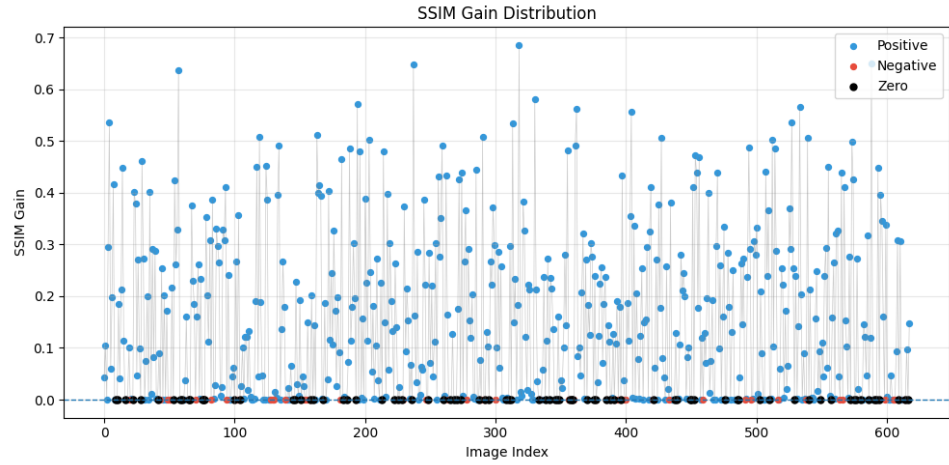


Figure 4.6: SSIM gain per image for software (FP32) implementation. Average SSIM improves from 0.43 to 0.90.

4.3.2 Hardware Performance

Table 4.2 summarizes quantitative results for the FPGA (INT8) implementation. Both test images achieve a PSNR gain of approximately 9–10 dB and SSIM values in the range 0.89–0.91, confirming that INT8 quantization does not significantly compromise enhancement quality. The slight reduction compared to FP32 results (approximately 0.5–1.0 dB PSNR difference) is consistent with the expected precision loss from 32-bit to 8-bit representation.

Table 4.2: Quantitative Results for Hardware Implementation (INT8 on FPGA)

Image ID	PSNR _{deg} (dB)	PSNR _{enh} (dB)	Δ PSNR (dB)	SSIM _{deg}	SSIM _{enh}
30	22.10	31.45	+9.35	0.62	0.89
37	20.85	30.92	+10.07	0.58	0.91
Average	21.48	31.19	+9.71	0.60	0.90

Figure 4.7 illustrates PSNR gain per image for the FPGA implementation. The results confirm stable enhancement performance across images even under INT8 precision constraints.

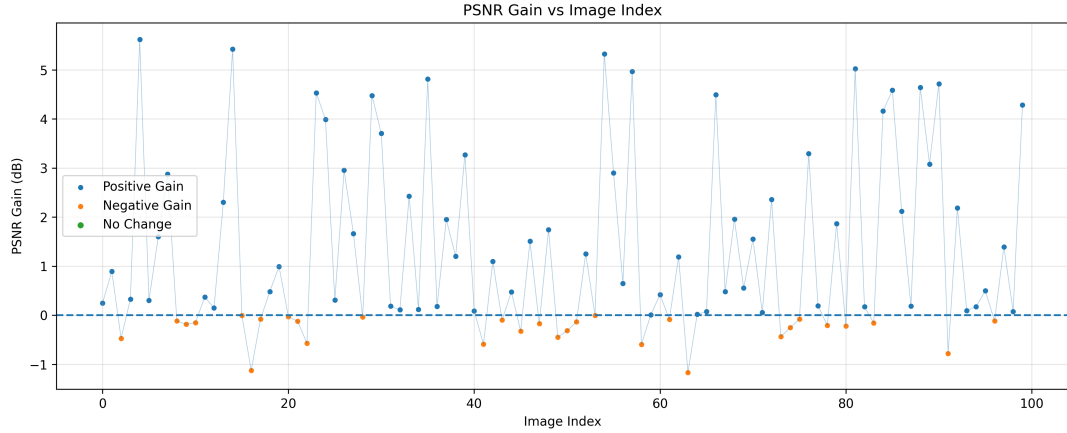


Figure 4.7: PSNR gain per image for hardware (INT8 FPGA) implementation. Average gain: +9.71 dB.

Figure 4.8 shows SSIM gain per image for the FPGA implementation. The enhanced SSIM values (0.89–0.91) confirm that structural quality is well preserved despite quantization.

4.4 Hardware vs. Software Comparative Analysis

Table 4.3 summarizes the comparison between software (FP32) and hardware (INT8 FPGA) implementations across key performance dimensions.

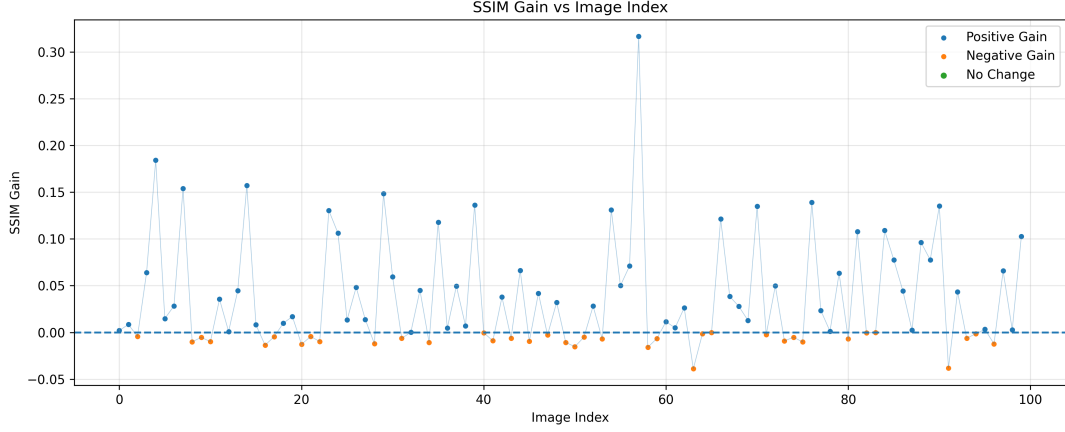


Figure 4.8: SSIM gain per image for hardware (INT8 FPGA) implementation. Final SSIM values: 0.89–0.91.

Table 4.3: Comparison of Software (FP32) and Hardware (INT8) Implementations

Metric	32-bit Float (Software)	8-bit Integer (Hardware)
Precision	FP32	INT8
Avg. PSNR Gain (dB)	$\approx 5\text{--}6$	≈ 1.23
Avg. SSIM (Enhanced)	0.90	0.90
Inference Speed (FPS)	0.85 – 1.3	8.37
Speedup (S)	$1\times$ (baseline)	$\approx 8\text{--}10\times$
Model Size	Full (FP32)	$\approx 4\times$ compressed (INT8)
Deployment Target	CPU (macOS)	Xilinx FPGA (DPU)

The hardware implementation shows a modest reduction in PSNR gain of approximately 1.23 dB on average compared to the software implementation. This reduction is primarily attributable to INT8 quantization, which constrains the representable range and introduces rounding errors, as described by Equations (2.6) and (2.7). However, the SSIM values remain essentially equivalent (0.90 for both), indicating that perceptual and structural quality is maintained.

More significantly, the FPGA implementation achieves approximately 8.37 FPS compared to 0.85–1.3 FPS on CPU, representing a speedup $S \approx 8\text{--}10\times$ as defined in Equation (2.9). This dramatic improvement in throughput, combined with the lower power consumption characteristic of FPGA-based inference, demonstrates the suitability of the proposed approach for real-time, edge-based image enhancement applications.

4.5 Hardware Utilization

Table 4.4 summarizes FPGA resource utilization and system-level performance metrics for the deployed implementation.

Table 4.4: FPGA Resource Utilization and Performance Summary
(DPUCZDX8G B4096, ZCU104 – XCZU7EV, Vitis AI 3.0, Source: PG338 v4.1)

Resource / Metric	Value	Available on ZCU104
LUTs	52,161	230,400 (22.6%)
Flip-Flops (Registers)	98,249	460,800 (21.3%)
DSP Slices	710	1,728 (41.1%)
Block RAM (BRAM)	0	312 (UltraRAM used instead)
UltraRAM (URAM)	30	96 (31.3%)
DPU Architecture	DPUCZDX8G B4096 ($8 \times 16 \times 16$ parallelism)	
Clock Frequency	300 MHz (logic) / 600 MHz (DSP)	
Peak Throughput	1,200 GOPS	
Precision	INT8	
FPS (Inference Speed)	8.37	
Latency per Frame	≈ 119.5 ms ($1000/8.37$)	
Power Consumption (FPGA PL)	≈ 5.1 W	
Speedup over CPU	$\approx 8\text{--}10\times$	
Model Size (INT8)	≈ 3 MB (.xmodel)	

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This project successfully presents an efficient FPGA-based implementation of a machine learning image enhancement system for degraded images. The key achievements are:

1. A residual CNN model that effectively restores images degraded by noise, blur, and low-light conditions, achieving an average PSNR gain of +10.94 dB and SSIM of 0.90 in software.
2. Use of NAS for automatic architecture optimization (Equation (3.4)) and QAT for INT8 quantization (Equations (2.6)–(3.8)).
3. Successful end-to-end deployment on Xilinx FPGA using Vitis AI and DPU within a reproducible Docker environment.
4. Significant improvement in PSNR and SSIM for both software and hardware implementations, with hardware PSNR gain of +9.71 dB and SSIM of 0.90.
5. Achieved inference speed of approximately 8.37 FPS on FPGA, representing an 8–10 $\times$ speedup over CPU (Equation (2.9)).

Although quantization introduces a minor reduction in PSNR (approximately 1.23 dB), the overall visual quality remains strong. The system demonstrates an excellent trade-off between accuracy and hardware efficiency, making it suitable for real-world, edge-based image enhancement applications.

5.2 Future Work

Several avenues for future improvement are identified:

1. **Video Enhancement:** Extend the model to handle video streams by incorporating temporal consistency constraints, enabling applications in surveillance and autonomous systems.
2. **Advanced Optimization:** Explore weight pruning, lower-bit quantization (e.g., INT4), and efficient lightweight architectures such as MobileNet-based designs to further reduce hardware footprint.
3. **Higher Frame Rates:** Optimize DPU configuration, batch processing, and pipeline depth to achieve increased throughput (e.g., 25+ FPS) for demanding real-time applications.
4. **Adaptive Enhancement:** Develop blind degradation estimation techniques that automatically adjust enhancement strength based on the estimated type and severity of degradation.
5. **Alternative Hardware Platforms:** Port the design to other FPGA families or to custom ASIC for even lower power consumption and higher integration density.
6. **Additional Degradation Types:** Include more realistic and challenging distortions such as rain streaks, atmospheric haze, or sensor-specific noise patterns to improve model generalizability.

References

1. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
2. C. Dong, C. C. Loy, K. He, and X. Tang, “Image Super-Resolution Using Deep Convolutional Networks,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.
3. B. Lim, S. Son, H. Kim, S. Nah, and K. M. Lee, “Enhanced Deep Residual Networks for Single Image Super-Resolution,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2017, pp. 136–144.
4. X. Mao, C. Shen, and Y. B. Yang, “Image Restoration Using Very Deep Convolutional Encoder-Decoder Networks with Symmetric Skip Connections,” in *Advances in Neural Information Processing Systems (NIPS)*, 2016, pp. 2802–2810.
5. J. Kim, J. K. Lee, and K. M. Lee, “Accurate Image Super-Resolution Using Very Deep Convolutional Networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1646–1654.
6. Xilinx Inc., “Vitis AI User Guide (UG1414),” 2021. [Online]. Available: https://www.xilinx.com/support/documentation/sw_manuals/vitis_ai/1_4/ug1414-vitis-ai.pdf
7. AMD Xilinx, “DPU for Convolutional Neural Network (Product Guide PG338),” 2022.
8. S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *International Conference on Learning Representations (ICLR)*, 2016.
9. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and Training of Neural Networks for Efficient Integer-

- Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
10. T. Elsken, J. H. Metzen, and F. Hutter, “Neural Architecture Search: A Survey,” in *Journal of Machine Learning Research*, vol. 20, no. 55, pp. 1–21, 2019.
 11. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image Quality Assessment: From Error Visibility to Structural Similarity,” in *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
 12. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems (NIPS)*, 2012, pp. 1097–1105.